AI PERSONAL STUDY ASSISTANT

Study Material PDF

Subject: Python Programming – Beginner to Intermediate

Department: Computer Science and Engineering

Prepared for: __IBM skillsbuild____
Name : sulthan faiz___
Institution: ___Tkmit_____

_____

A concise learning guide with explanations, examples, revision questions, and practice exercises.

1. Introduction to Python

What is Python? Python is a high-level, general-purpose programming language. Its readable syntax makes it suitable for beginners as well as professional developers.

Why learn Python?

- It has simple and readable syntax.
- It is used in web development, automation, data science, artificial intelligence, and machine learning.
- It has a large collection of libraries and a helpful community.
- It supports procedural, object-oriented, and functional programming styles.

Writing and running a program

A Python program is usually saved with the **.py** extension. You can run it with a Python interpreter or through an editor such as IDLE, VS Code, or an online Python environment.

Example: your first program

```
print("Hello, World!")
```

The **print()** function displays information on the screen. Text is written inside quotation marks.

Basic programming workflow

1. Understand the problem. 2. Plan the steps. 3. Write the code. 4. Run the program. 5. Test the result. 6. Fix errors and improve the solution.

2. Variables, Data Types, and Operators

A **variable** is a name used to refer to a value. Python creates a variable when you assign a value to it.

Examples of variables

```
name = "Anu"
age = 18
height = 1.72
is_student = True
```

Common built-in data types

Type	Example	Meaning
int	25	Whole number
float	3.14	Decimal number
str	"Python"	Text
bool	True / False	Logical value
list	[1, 2, 3]	Ordered, changeable collection
tuple	(1, 2, 3)	Ordered, unchangeable collection
dict	{"id": 1}	Key-value pairs

Operators

Arithmetic operators perform calculations: **+** addition, **-** subtraction, ***** multiplication, / division, // floor division, **%** remainder, and ****** exponentiation.

Example

```
a = 10
b = 3
print(a + b) # 13
print(a % b) # 1
print(a ** b) # 1000
```

3. Decisions, Loops, and Functions

Conditional statements

An **if** statement lets a program choose what to do based on a condition. Indentation defines the block of code.

```
mark = 72
if mark >= 50:
    print("Pass")
else:
    print("Try again")
```

Loops

A **for** loop repeats over a sequence or range. A **while** loop repeats while its condition is true.

```
for number in range(1, 4):
    print(number)

count = 1
while count <= 3:
    print(count)
    count += 1
```

Functions

A function is a reusable block of code. Define it with **def**. Parameters provide input, and **return** sends a result back to the caller.

```
def add(x, y):
    return x + y

answer = add(4, 6)
print(answer) # 10
```

Good practice

Use meaningful names, keep functions focused on one task, indent consistently, and test boundary cases such as zero, negative values, and empty input.

4. Collections, Errors, and Files

Lists and dictionaries

A list stores multiple items in order and can be changed. A dictionary stores values using keys.

```
subjects = ["Python", "Maths", "Physics"]
subjects.append("English")
print(subjects[0]) # Python

student = {"name": "Maya", "mark": 85}
print(student["name"]) # Maya
```

Handling errors

Exceptions can occur while a program runs. A **try**/**except** block allows a program to handle expected errors without stopping abruptly.

```
try:
    number = int(input("Enter a number: "))
    print(100 / number)
except ValueError:
    print("Please enter a valid integer.")
except ZeroDivisionError:
    print("The number must not be zero.")
```

Reading a text file

```
with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
print(content)
```

The **with** statement helps close a file automatically. Use the correct file mode: **r** for reading, **w** for writing (overwrites existing content), and **a** for appending.

Debugging checklist

Read the full error message; check spelling and indentation; verify variable values; test a small example; and change one thing at a time.

5. Revision Questions and Practice

Quick revision questions

1. What is Python, and mention two areas where it is used.
2. What is the difference between an integer and a float?
3. What is the purpose of an if/else statement?
4. Explain the difference between a for loop and a while loop.
5. Why are functions useful in a program?
6. What is the difference between a list and a tuple?
7. Why should exceptions be handled?

Practice exercises

- Write a program to add two numbers entered by the user.
- Write a program that checks whether a number is even or odd.
- Print the multiplication table of a number using a loop.
- Create a function that returns the largest of three numbers.
- Store five subject marks in a list and calculate their average.

Answer hints

Even/odd: use **number % 2**. Average: divide the sum of marks by the number of marks. Largest value: compare the three inputs or use the built-in **max()** function.

Suggested study method

1. Read one topic. 2. Explain it in your own words. 3. Type and run each example. 4. Answer the revision questions without looking. 5. Complete the exercises. 6. Ask your AI study assistant to explain mistakes using this PDF.

End of study material